Experiment 1: Set up Qiskit and implement basic quantum gates

Cell1:

```
%pip install --upgrade pip
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
# Import the quantum circuit class
from qiskit import QuantumCircuit

# Import transpile and Aer simulator
from qiskit import transpile
from qiskit_aer import Aer

# Import Bloch sphere visualization
from qiskit.visualization import plot_bloch_multivector

print("Libraries imported successfully!")
```

cell3:

```
# Create a quantum circuit with 1 qubit
qc = QuantumCircuit(1)
# Apply Pauli-X gate (NOT gate)
qc.x(0)
# Apply Pauli-Z gate (phase-flip gate)
qc.z(0)
# Apply Hadamard gate
qc.h(0)
# Display the quantum circuit
print("Quantum Circuit:")
qc.draw("mpl")
```

cell4:

```
# Select the statevector simulator
backend = Aer.get_backend("statevector_simulator")
# Compile and execute the circuit
result = backend.run(
   transpile(qc, backend)
).result()
# Get the final statevector
psi = result.get_statevector()
```

```
print("Simulation completed successfully!")


cell5:
print("Final Quantum State:")
print(psi)

# Visualize the final state on the Bloch sphere
plot_bloch_multivector(psi)
```

Experiment 2: Construct a quantum circuit applying H → S → H → T on a single qubit |1>

Cell1:

```
%pip install qiskit

%pip install qiskit-aer

%pip install pylatexenc
```

Cell2:

```
from qiskit import QuantumCircuit, transpile

from qiskit_aer import AerSimulator, Aer

from qiskit.visualization import plot_histogram

from IPython.display import display

print("Libraries imported successfully!")
```

cell3:

```
# Create a circuit with 1 qubit and 1 classical bit

qc = QuantumCircuit(1, 1)


# Initialize the qubit to |1>

qc.x(0)


# Apply H -> S -> H -> T
```

```
qc.h(0)
qc.s(0)
qc.h(0)
qc.t(0)

# Display the circuit
display(qc.draw("mpl"))

cell4:
# Create a circuit without classical bits or measurements
sv_circuit = QuantumCircuit(1)

sv_circuit.x(0)
sv_circuit.h(0)
sv_circuit.s(0)
sv_circuit.h(0)
sv_circuit.t(0)

# Select the statevector simulator
backend = Aer.get_backend("statevector_simulator")

# Execute the circuit
```

```
result = backend.run(
    transpile(sv_circuit, backend)
).result()

# Obtain the final statevector
psi = result.get_statevector()
print("Final Statevector:")
print(psi)

cell5:
# Measure qubit 0 into classical bit 0
qc.measure(0, 0)
# Display the circuit with measurement
display(qc.draw("mpl"))

cell6:
# Create the simulator
simulator = AerSimulator()
# Compile the circuit
compiled_circuit = transpile(qc, simulator)

# Run 1000 shots
```

```
result = simulator.run(
    compiled_circuit,
    shots=1000
).result()
# Obtain measurement counts
counts = result.get_counts()
print("Measurement Counts:")
print(counts)

cell7:
plot_histogram(counts)

cell8:
total_shots = sum(counts.values())

probabilities = {
state: count / total_shots
    for state, count in counts.items()
}

print("Output Probabilities:")
print(probabilities
```

Experiment 3: Generate the Bell State $|\Phi^+\rangle$ using a Two-Qubit Quantum Circuit

1:

```
%pip install matplotlib

%pip install qiskit

%pip install qiskit-aer

%pip install pylatexenc
```

Cell2:

```
from qiskit import QuantumCircuit, transpile

from qiskit_aer import Aer, AerSimulator


from qiskit.visualization import (
    plot_histogram,
    plot_state_city,
    plot_state_hinton,
    plot_state_qsphere
)


from IPython.display import display


print("Libraries imported successfully!")
```

cell3:

```
# Create a quantum circuit with 2 qubits and 2 classical bits
bell = QuantumCircuit(2, 2)
# Apply Hadamard gate to qubit 0
bell.h(0)
# Apply CNOT gate with qubit 0 as control and qubit 1 as target
bell.cx(0, 1)
# Measure both qubits
bell.measure([0, 1], [0, 1])
# Display the circuit
display(bell.draw("mpl"))
```

cell4:

```
# Create the simulator
simulator = AerSimulator()
# Compile the circuit
compiled = transpile(bell, simulator)
# Run 1000 shots
result = simulator.run(
    compiled,
    shots=1000
).result()
```

```
# Get measurement counts
counts = result.get_counts()
print("Measurement Counts:")
print(counts)

cell5:
plot_histogram(counts)

cell6:
bell_sv = QuantumCircuit(2)
# Hadamard gate
bell_sv.h(0)
# CNOT gate
bell_sv.cx(0, 1)
display(bell_sv.draw("mpl"))

cell7:
# Select the statevector simulator
backend = Aer.get_backend("statevector_simulator")
# Execute the circuit
result = backend.run(
   transpile(bell_sv, backend)
```

```
).result()
# Obtain the statevector
psi = result.get_statevector()
print("Final Statevector:")
print(psi)

cell8:
plot_state_city(psi)

cell9:
plot_state_hinton(psi)

cell10:
plot_state_qsphere(psi)
```

Experiment 4: Generate the Bell State |phi->

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator, Aer
from qiskit.visualization import (
    plot_histogram,
    plot_state_paulivec,
    plot_state_hinton
)

from IPython.display import display
print("Libraries imported successfully!")
```

cell3:

```
# Create a quantum circuit with 2 qubits
qc = QuantumCircuit(2)
```

```
# Apply Hadamard gate
qc.h(0)
# Apply CNOT gate
qc.cx(0, 1)
# Add Z gate to generate |Phi->
qc.z(0)
# Display the circuit
display(qc.draw("mpl"))
```

cell4:

```
# Select the statevector simulator
backend = Aer.get_backend("statevector_simulator")
# Run the circuit
result = backend.run(
    transpile(qc, backend)
).result()
# Obtain the statevector
psi = result.get_statevector()
print("Final Statevector:")
print(psi)
```

cell5: plot_state_paulivec(psi)

cell6:

```
plot_state_hinton(psi)
```

cell7:

```
# Create a new circuit with 2 qubits and 2 classical bits
qc_measure = QuantumCircuit(2, 2)
# Generate the Bell state |Phi->
qc_measure.h(0)
qc_measure.cx(0, 1)
qc_measure.z(0)
# Measure both qubits
qc_measure.measure([0, 1], [0, 1])
# Display the circuit
display(qc_measure.draw("mpl"))
```

cell 8:

```
# Create simulator
simulator = AerSimulator()

# Compile the circuit
compiled_circuit = transpile(qc_measure, simulator)
```

```
# Run 1000 shots
result = simulator.run(
    compiled_circuit,
    shots=1000
).result()

# Get measurement counts
counts = result.get_counts()
print("Measurement Counts:")
print(counts)



cell9:
plot_histogram(counts)
```

Exp 5: Generate the Bell State |psi+>

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator, Aer
from qiskit.visualization import plot_histogram, plot_state_city
from IPython.display import display
```

cell3:

```
# Create a circuit with 2 qubits
qc = QuantumCircuit(2)
# Apply Hadamard gate to qubit 0
qc.h(0)
# Apply CNOT gate to entangle both qubits
qc.cx(0, 1)
# Apply Pauli-X gate to qubit 1 to generate |Psi+>
qc.x(1)
```

```
# Display the circuit
display(qc.draw("mpl"))
```

cell4:

```
# Select the statevector simulator
backend = Aer.get_backend("statevector_simulator")
# Compile and execute the circuit
result = backend.run(transpile(qc, backend)).result()
# Obtain and display the final statevector
psi = result.get_statevector()
print("Final Statevector:")
print(psi)
```

cell5:

```
# Display the real and imaginary
#parts of the density matrix
plot_state_city(psi)
```

cell6:

```
# Create a circuit with 2 qubits and 2 classical bits
qc_measure = QuantumCircuit(2, 2)
# Generate the Bell state |Psi+>
```

```
qc_measure.h(0)
qc_measure.cx(0, 1)
qc_measure.x(1)
# Measure both qubits
qc_measure.measure([0, 1], [0, 1])
# Display the circuit with measurements
display(qc_measure.draw("mpl"))

cell7:
# Create the Aer simulator
sim = AerSimulator()
# Compile and run the circuit for 1024 shots
result = sim.run(
    transpile(qc_measure, sim),
    shots=1024
).result()
# Get and print measurement counts
counts = result.get_counts()
print("Measurement Counts:", counts)

cell8:
plot_histogram(counts)
```

Experiment 10: Implement a 3-Qubit Circuit Using SWAP with 3 CNOT Gates

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
# Import Qiskit circuit and simulator tools
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
```

cell3:

```
# Create a circuit with 3 qubits and 3 classical bits
qc = QuantumCircuit(3, 3)
# Prepare the input state |101>
qc.x(0)
qc.x(2)
```

```
# Display the initial circuit
display(qc.draw("mpl"))

cell4:
# Swap qubit 0 and qubit 1 using three CNOT gates
qc.cx(0, 1)
qc.cx(1, 0)
qc.cx(0, 1)
# Display the circuit
display(qc.draw("mpl"))

cell5:
# Measure each qubit into its corresponding classical bit
qc.measure([0, 1, 2], [0, 1, 2])
# Display the complete circuit
display(qc.draw("mpl"))

cell6:
# Create the Aer simulator
simulator = AerSimulator()
# Compile and run the circuit
compiled = transpile(qc, simulator)
```

```
result = simulator.run(compiled, shots=1024).result()
# Display measurement counts
counts = result.get_counts()
print("Measurement Counts:", counts)
```

cell7:

```
# Display the measurement results
plot_histogram(counts)
```

Cell 8: Test input \(|010\rangle\)

```
# Create a new circuit for input |010>
qc2 = QuantumCircuit(3, 3)

# Prepare the input state |010>
qc2.x(1)

# Swap qubit 0 and qubit 1 using three CNOT gates
qc2.cx(0, 1)
qc2.cx(1, 0)
qc2.cx(0, 1)
# Measure all qubits
qc2.measure([0, 1, 2], [0, 1, 2])
```

```
# Display the circuit
display(qc2.draw("mpl"))


cell9:
# Compile and execute the second circuit
compiled2 = transpile(qc2, simulator)
result2 = simulator.run(compiled2, shots=1024).result()
# Print measurement counts
counts2 = result2.get_counts()
print("Measurement Counts for |010>:", counts2)
# Plot the histogram
plot_histogram(counts2)
```

Experiment 11: Superdense Coding — Send Classical Bits 01

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
# Import Qiskit circuit and simulator tools
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
```

cell3:

```
# Create a quantum circuit with 2 qubits
qc = QuantumCircuit(2, 2)
# Generate a shared Bell pair
qc.h(0)
qc.cx(0, 1)
```

```
# Add a barrier after Bell pair creation
qc.barrier()
# Display the circuit
display(qc.draw("mpl"))
```

cell4:

```
# For 01, apply the Z gate to qubit 0
qc.z(0)
# Add a barrier after encoding
qc.barrier()
# Display the circuit
display(qc.draw("mpl"))
```

cell5:

```
# Bob decodes the message using CNOT and Hadamard gates
qc.cx(0, 1)
qc.h(0)
# Measure both qubits
qc.measure([0, 1], [0, 1])
# Display the complete circuit
display(qc.draw("mpl"))
```

cell6:

```
# Create the Aer simulator
simulator = AerSimulator()
# Compile and execute the circuit for 1024 shots
result = simulator.run(
    transpile(qc, simulator),
    shots=1024
).result()
# Get and print measurement counts
counts = result.get_counts()
print("Alice sent: 01")
print("Bob received:", counts)
```

cell7:

```
# Display the measurement results
plot_histogram(counts)
```

Experiment 12: Superdense Coding — Send Classical Bits 10

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
# Import Qiskit circuit and simulator tools
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

# Import histogram visualization
from qiskit.visualization import plot_histogram
from IPython.display import display
```

cell3:

```
# Create a circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)
# Generate a shared Bell pair
qc.h(0)
```

```
qc.cx(0, 1)
# Separate circuit stages
qc.barrier()
# Display the circuit
display(qc.draw("mpl"))
```

c4:

```
# Alice encodes message 10 by applying X to qubit 0
qc.x(0)
# Separate encoding and decoding stages
qc.barrier()
# Display the circuit
display(qc.draw("mpl"))
```

cell5:

```
# Bob decodes using CNOT and Hadamard gates
qc.cx(0, 1)
qc.h(0)
# Measure both qubits
qc.measure([0, 1], [0, 1])
# Display the complete circuit
display(qc.draw("mpl"))
```

cell6:

```
simulator = AerSimulator()
# Compile and execute for 1024 shots
result = simulator.run(
    transpile(qc, simulator),
    shots=1024
).result()

# Get measurement counts
counts = result.get_counts()
# Display results
print("Alice sent: 10")
print("Bob received:", counts)
```

cell7:

```
# Display measurement outcomes
plot_histogram(counts)
```

Experiment 13: Superdense Coding — Send Classical Bits 11

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Celll2:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

from qiskit.visualization import plot_histogram
from IPython.display import display
```

cell3:

```
# Create a circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)
# Generate a shared Bell pair
qc.h(0)
qc.cx(0, 1)
# Separate circuit stages
```

```
qc.barrier()
# Display the circuit
display(qc.draw("mpl"))
```

cell4:

```
# Alice encodes message 11 by applying Z and X gates
qc.z(0)
qc.x(0)
# Separate encoding and decoding stages
qc.barrier()
# Display the circuit
display(qc.draw("mpl"))
```

cell5:

```
# Bob decodes using CNOT and Hadamard gates
qc.cx(0, 1)
qc.h(0)
# Measure both qubits
qc.measure([0, 1], [0, 1])
# Display the complete circuit
display(qc.draw("mpl"))
```

cell6:

```
# Create the Aer simulator
simulator = AerSimulator()
# Compile and execute for 1024 shots
result = simulator.run(
    transpile(qc, simulator),
    shots=1024
).result()
# Get measurement counts
counts = result.get_counts()
# Display results
print("Alice sent: 11")
print("Bob received:", counts)
```

cell7:

```
# Display measurement outcomes
plot_histogram(counts)
```

Experiment 14: Deutsch–Jozsa Algorithm for a Balanced Function with 3 Input Qubits

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
# Import NumPy and Qiskit tools
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
import matplotlib.pyplot as plt
```

cell3:

```
# Create an oracle for a balanced function
def dj_oracle(n):
    oracle = QuantumCircuit(n + 1, name="Oracle")
```

```
    # Apply CNOT gates to create a balanced function
    for q in range(n):
        oracle.cx(q, n)

    return oracle
```

cell4:

```
# Create the Deutsch–Jozsa circuit
def deutsch_jozsa(n):
    qc = QuantumCircuit(n + 1, n)

    # Prepare the ancilla qubit in state |1>
    qc.x(n)
    qc.h(n)

    # Apply Hadamard gates to input qubits
    for q in range(n):
        qc.h(q)

    qc.barrier()

    # Apply the balanced oracle
```

```
    oracle = dj_oracle(n)
    qc.compose(oracle, range(n + 1), inplace=True)

    qc.barrier()

    # Apply Hadamard gates again to input qubits
    for q in range(n):
        qc.h(q)

    # Measure the input qubits
    qc.measure(range(n), range(n))

    return qc

cell5:
# Use 3 input qubits
n = 3

# Create the circuit
qc = deutsch_jozsa(n)

# Display the circuit
```

```
display(qc.draw("mpl"))

# Create the simulator
simulator = AerSimulator()

# Compile and execute for 1024 shots
compiled = transpile(qc, simulator)
result = simulator.run(compiled, shots=1024).result()

# Display measurement counts
counts = result.get_counts()
print("Measurement Counts:", counts)

cell6:
# Display measurement outcomes
plot_histogram(counts)

cell7:
# Check whether the measured result is all zeros
most_likely = max(counts, key=counts.get)

if most_likely == "000":
```

```
    print("Result: ORACLE IS CONSTANT")
else:
    print("Result: ORACLE IS BALANCED")
```

Experiment 18: Deutsch–Jozsa Algorithm for Constant and Balanced Functions

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell 2:

```
# Import required libraries
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
import matplotlib.pyplot as plt
```

cell3:

```
# Create an oracle for constant or balanced functions
def dj_oracle(case, n):
    oracle = QuantumCircuit(n + 1, name="Oracle")
```

```
    if case == "constant":
        # Constant function f(x) = 0
        pass

    elif case == "balanced":
        # Balanced function f(x) = x0
        oracle.cx(0, n)

    else:
        raise ValueError("Case must be constant or balanced")

    return oracle
```

cell 4:

```
# Create the Deutsch–Jozsa circuit
def dj_algorithm(oracle, n):
    qc = QuantumCircuit(n + 1, n)

    # Prepare ancilla qubit in state |1>
    qc.x(n)
    qc.h(n)
```

```
# Apply Hadamard gates to input qubits
for q in range(n):
    qc.h(q)

qc.barrier()

# Apply the oracle
qc.compose(oracle, range(n + 1), inplace=True)

qc.barrier()

# Apply Hadamard gates again
for q in range(n):
    qc.h(q)

# Measure input qubits
qc.measure(range(n), range(n))

return qc
```

cell5:

```
# Select two input qubits
n = 2

# Choose the oracle type: constant or balanced
case = input("Enter oracle type (constant/balanced): ").strip().lower()

# Create the oracle and circuit
oracle = dj_oracle(case, n)
qc = dj_algorithm(oracle, n)

# Display the circuit
print("Deutsch-Jozsa Circuit:")
display(qc.draw("mpl"))

# Simulate the circuit
simulator = AerSimulator()
compiled = transpile(qc, simulator)
result = simulator.run(compiled, shots=1024).result()

# Display measurement counts
counts = result.get_counts()
```

```
print("Measurement Counts:", counts)

cell6:
# Display the measurement histogram
display(plot_histogram(counts))

cell7:
# Determine whether the function is constant or balanced
most_likely = max(counts, key=counts.get)

if most_likely == "00":
    print("Result: ORACLE IS CONSTANT")
else:
    print("Result: ORACLE IS BALANCED")
```

Experiment 19: Implement Grover's Algorithm to Search for the Marked State |11>

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display
```

cell3:

```
# Create a circuit with 2 qubits and 2 classical bits
qc = QuantumCircuit(2, 2)

# Apply Hadamard gates to create a superposition
qc.h(0)
qc.h(1)
```

```
# Display the circuit
display(qc.draw("mpl"))
```

cell4:

```
# Mark the target state |11> using a controlled-Z gate
qc.cz(0, 1)
# Display the circuit
display(qc.draw("mpl"))
```

cell5:

```
# Apply Hadamard gates
qc.h(0)
qc.h(1)
# Apply X gates
qc.x(0)
qc.x(1)
# Apply controlled-Z gate
qc.cz(0, 1)
# Apply X gates again
qc.x(0)
qc.x(1)
# Apply Hadamard gates again
qc.h(0)
```

```
qc.h(1)
# Display the circuit
display(qc.draw("mpl"))

cell6:
# Measure both qubits
qc.measure([0, 1], [0, 1])
# Display the complete circuit
display(qc.draw("mpl"))

cell7:
# Create the Aer simulator
simulator = AerSimulator()
# Compile and execute the circuit for 1024 shots
compiled = transpile(qc, simulator)
result = simulator.run(compiled, shots=1024).result()
# Get and display measurement counts
counts = result.get_counts()
print("Measurement Counts:", counts)

cell8:
plot_histogram(counts)
```

Experiment 20: Shor's Algorithm for Factorization of a User-Entered Number

Cell1:

```
%pip install matplotlib
%pip install qiskit
%pip install qiskit-aer
%pip install pylatexenc
```

Cell2:

```
# Import required libraries
from math import gcd
from fractions import Fraction
from random import randrange
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

print("Libraries imported successfully!")
```

cell3:

```
# Ask the user to enter the number to factorize
N = int(input("Enter a composite number N: "))

# Validate the input
if N < 4:
    raise ValueError("Enter a composite integer greater than or equal to 4.")

print("Number entered:", N)
```

Orcell3:

```
# Automatically search for nontrivial factors of N
def shor_factor_demo(N):
    # Handle even numbers directly
    if N % 2 == 0:
        print(f"Factors of {N}: 2 and {N // 2}")
        return

    for a in range(2, N):
        g = gcd(a, N)
```

```
# A GCD may reveal a factor immediately
if 1 < g < N:
    print(f"Factors of {N}: {g} and {N // g}")
    return

# Skip bases that are not coprime to N
if g != 1:
    continue

# Find the period
r = 1
while pow(a, r, N) != 1:
    r += 1

print(f"Trying a={a}, period r={r}")

# The period must be even
if r % 2 != 0:
    continue

value = pow(a, r // 2, N)
```

```
        # Avoid trivial cases
        if value in (1, N - 1):
            continue

        # Calculate candidate factors
        p = gcd(value - 1, N)
        q = gcd(value + 1, N)

        if 1 < p < N and 1 < q < N:
            print(f"Factors of {N}: {p} and {q}")
            return

    print("No nontrivial factors found.")

# Run the factorization
shor_factor_demo(N)
```

Cell4:

```
# Import plotting library
import matplotlib.pyplot as plt

# Enter a base coprime to N
a = int(input(f"Enter a base a (2 to {N-1}): "))

# Check whether a is valid
if not 1 < a < N or gcd(a, N) != 1:
    raise ValueError("Choose a between 2 and N-1 that is coprime to N.")

# Calculate modular exponentiation values
x_values = list(range(2 * N))
y_values = [pow(a, x, N) for x in x_values]

# Plot the periodic function
plt.plot(x_values, y_values, marker="x", linestyle="dotted")
plt.xlabel("x")
plt.ylabel(f"{a}^x mod {N}")
plt.title("Periodic Function in Shor's Algorithm")
plt.grid(True)
plt.show()
```

cell5:

```
# Find the smallest positive r such that a^r mod N = 1
r = 1

while pow(a, r, N) != 1:
    r += 1

print("Base a =", a)
print("Period r =", r)
```

cell6:

```
# Check whether the period is even
if r % 2 != 0:
    print("Period is odd. Choose another base a and repeat.")
else:
    # Calculate a^(r/2) modulo N
    value = pow(a, r // 2, N)

    # Calculate possible factors using GCD
    factor1 = gcd(value - 1, N)
    factor2 = gcd(value + 1, N)
```

```
print("Possible factor 1:", factor1)
print("Possible factor 2:", factor2)

# Display nontrivial factors
factors = sorted({
    f for f in (factor1, factor2)
    if 1 < f < N and N % f == 0
})

if factors:
    print("Nontrivial factors found:", factors)
    for f in factors:
        print(f"{N} = {f} x {N // f}")
else:
    print("No nontrivial factors found. Try another base a.")
```